StudyNest

AI Development & Implementation Playbook

Purpose: Tell an AI coding agent how to build StudyNest incrementally, safely, and professionally instead of generating the whole application in one uncontrolled pass.

1. Core Rule

Never ask the AI to build the entire project at once. Build one milestone/feature at a time. Inspect the diff, run the app, test the feature, then continue.

The project specification is the source of truth. The AI is an implementation assistant, not the product owner.

2. AI Operating Principles

- Read the existing repository before modifying it.
- Understand and reuse the current architecture.
- Do not rewrite unrelated files.
- Do not add libraries without a clear requirement.
- Prefer simple, maintainable code.
- Use strict TypeScript and avoid unnecessary any.
- Separate business logic from presentation where practical.
- Validate input before persistence.
- Never expose secrets in frontend code or source control.
- Never weaken authentication or RLS to make a feature work.
- After meaningful changes, run typecheck/lint/tests/build as appropriate.
- If ambiguous, state the assumption before implementing.

3. Product Context

StudyNest is a student productivity platform. Its central loop is: course → goals/tasks → study session → notes → analytics.

The UI should feel calm, focused, professional, and product-oriented—not like a generic AI-generated dashboard.

4. Standard AI Workflow

```
Plan
↓
Inspect repository
↓
Implement ONE small feature
↓
```

```
Review generated code
↓
Run typecheck/lint/tests
↓
Run the app manually
↓
Fix issues
↓
Commit
↓
Next feature
```

5. Milestones

0 — Repository & Specification

- Inspect repository/tooling.
- Create a concise implementation plan.
- Confirm Vue 3, TypeScript, Vite, Tailwind, Pinia, Vue Router, Supabase.
- Do not implement product features yet.

1 — Foundation

- Set up app shell, layouts, router, Pinia, Supabase client, linting, formatting, shared UI primitives.
- Verify build.

2 — Authentication

- Register/login/logout/session persistence/profile.
- Protect app routes.
- Test refresh, logout, invalid credentials, unauthenticated navigation.

3 — Database

- Create all six core tables.
- Add foreign keys, timestamps, constraints, useful indexes.
- Implement and test RLS with multiple accounts.

4 — Courses

- Implement course CRUD, forms, list, detail page, delete confirmation.

5 — Tasks

- Implement CRUD, status, priority, deadlines, estimated time, search, filters, sorting, and states.

6 — Notes

- Implement bilingual English/Chinese notes, tags, course association, search, filtering, CRUD.

7 — Study Sessions

- Implement reliable start/pause/resume/finish timer.

- Persist completed sessions.
- Handle refresh/navigation safely.
- Calculate daily/weekly/monthly totals.

8 — Goals

- Implement target/current values, deadline, status, and safe progress calculations.

9 — Dashboard

- Aggregate real data into summary cards, today's tasks, active goals, recent sessions, course progress, weekly activity.

10 — Analytics

- Build charts from real data.
- Keep calculations in testable utilities.
- Provide useful empty states.

11 — UX & Accessibility

- Responsive layouts, dark mode, skeletons, empty/error/success states, toasts, dialogs, keyboard access, labels, focus states, semantic HTML.

12 — Testing & Deployment

- Test critical logic/forms.
- Run lint/typecheck/tests/build.
- Deploy and verify production auth/database behavior.

6. Feature Prompt Template

```
You are working on StudyNest, a Vue 3 + TypeScript + Supabase study management platform.

Before changing code:
1. Inspect the relevant existing files.
2. Explain the implementation plan briefly.
3. Identify any database/schema changes.
4. Implement only the requested feature.
5. Reuse existing components, stores, services, and types.
6. Do not modify unrelated files.

Requirements:
[FEATURE REQUIREMENTS]

Acceptance criteria:
[ACCEPTANCE CRITERIA]

After implementation:
- Run typecheck.
- Run lint.
- Run relevant tests.
- Run a production build if appropriate.
- Report changed files and assumptions.
```

7. Debugging Prompt Template

```
Investigate this StudyNest issue without rewriting unrelated code.

Problem:
[BUG]

Expected behavior:
[EXPECTED]

Actual behavior:
[ACTUAL]

Steps to reproduce:
[STEPS]

First inspect the relevant code and identify the likely root cause. Then make the smallest safe fix. Preserve the existing architecture and styling. After the fix, run relevant typecheck/tests/build and explain the root cause.
```

8. Code Quality Rules

- Use feature-oriented components; avoid giant Vue components.
- Use composables for reusable reactive behavior.
- Use Pinia for shared application state, not every local form field.
- Centralize Supabase access where practical.
- Define shared database/domain types.
- Prefer explicit interfaces and function types.
- Use async/await with deliberate error handling.
- Never silently swallow errors.
- Avoid duplicated fetching logic.
- Avoid magic numbers for business rules.
- Do not generate fake data to hide incomplete production functionality.

9. Database & Security Rules

- Every private table needs an authenticated ownership path.
- Enable and test RLS.
- Never rely only on frontend authorization.
- Use foreign keys for relationships.
- Use consistent timestamps.
- Add indexes based on real query patterns.
- Never expose service-role credentials in the browser.
- Treat user-provided content as untrusted input.

10. UI/UX Rules

- Use navy #172033, warm white #F8F9F7, white surfaces, emerald #16A36A, and soft mint #DDF5E9.
- Use Inter typography and Lucide icons.
- Use approximately 12px card radius and subtle borders.
- Avoid excessive gradients, glassmorphism, and oversized rounded containers.
- Every data-heavy page needs loading, empty, error, and success states.
- Destructive actions require confirmation.
- Forms need labels, validation, and clear errors.
- Responsive design is required from the beginning.
- Dark mode must use a deliberate dark palette.

11. Acceptance Checklist

- Feature works with real Supabase data.
- Auth behavior is correct.
- RLS/security behavior is verified.
- Desktop and mobile layouts work.
- Loading/empty/error/success states exist.
- TypeScript has no avoidable errors.
- Lint passes.
- Relevant tests pass.
- Production build passes.
- No secrets are committed.
- No unrelated files were changed unnecessarily.
- Documentation is updated when setup or behavior changes.

12. Git Strategy

```
main
├── feature/auth
├── feature/courses
├── feature/tasks
├── feature/notes
├── feature/study-sessions
├── feature/goals
└── feature/analytics

Example commits:
feat(auth): add protected route handling
feat(courses): implement course CRUD
feat(tasks): add task filtering
feat(analytics): add weekly study chart
```

13. What The AI Must Not Do

- Do not build the entire application in one response.
- Do not replace the chosen stack without a clear reason.
- Do not invent backend behavior unsupported by the schema.
- Do not disable RLS because of a permission error.
- Do not hardcode user-specific data in production screens.
- Do not create huge components when smaller ones are practical.
- Do not add unnecessary dependencies.
- Do not polish visuals while core functionality is broken.
- Do not claim a feature is complete without testing it.

14. Final Definition of Done

StudyNest is complete only when a user can register, create a course, create goals and tasks, write bilingual notes, run and save study sessions, see the dashboard update from real data, inspect analytics, and manage their account securely.

The UI must be responsive, accessible, polished, and consistent with the design system. The repository must build cleanly, critical logic must be tested, security policies must be verified, and the README must explain setup, architecture, schema, and deployment.

15. Build Order Summary

Foundation → Auth → Database/RLS → Courses → Tasks → Notes → Study Sessions → Goals → Dashboard → Analytics → UX Polish → Testing → Deployment → Documentation.